

Top-Mining

Архитектура системы

Платформа каталога майнинг-оборудования и калькулятора доходности. Nuxt-фронтенд, Nitro-BFF, Go GraphQL и PostgreSQL — с расчётом прибыли, который выполняется целиком на клиенте.

ПАРАМЕТР	ЗНАЧЕНИЕ
Слоёв	4 — Frontend · BFF · Backend · DB
Где считается прибыль	Только на клиенте (profit.ts)
Источник правды по данным	SQL-миграции (backend/migrations/)
Контракты API	GraphQL + Zod
Путь запроса	Browser → Nitro BFF → Go GraphQL → PostgreSQL

Содержание

01	Общая картина системы	3
02	Как устроен фронтенд	5
	Оболочка страницы	5
	Домены UI	5
	Компонентная диаграмма — калькулятор	6
03	Как устроен бэкэнд	6
04	GraphQL API + UML	7
	Queries	7
	Mutations	8
	UML — схема ↔ пакеты Go	8
05	Nuxt BFF · роуты server/api	8
	Zod-контракты	9
06	Схемы данных · ER	9
	Ядро калькулятора — hardware	9
	Заявки — leads	11
	Прочие домены	11
07	Sequence-диаграммы	11
	Загрузка каталога калькулятора	12

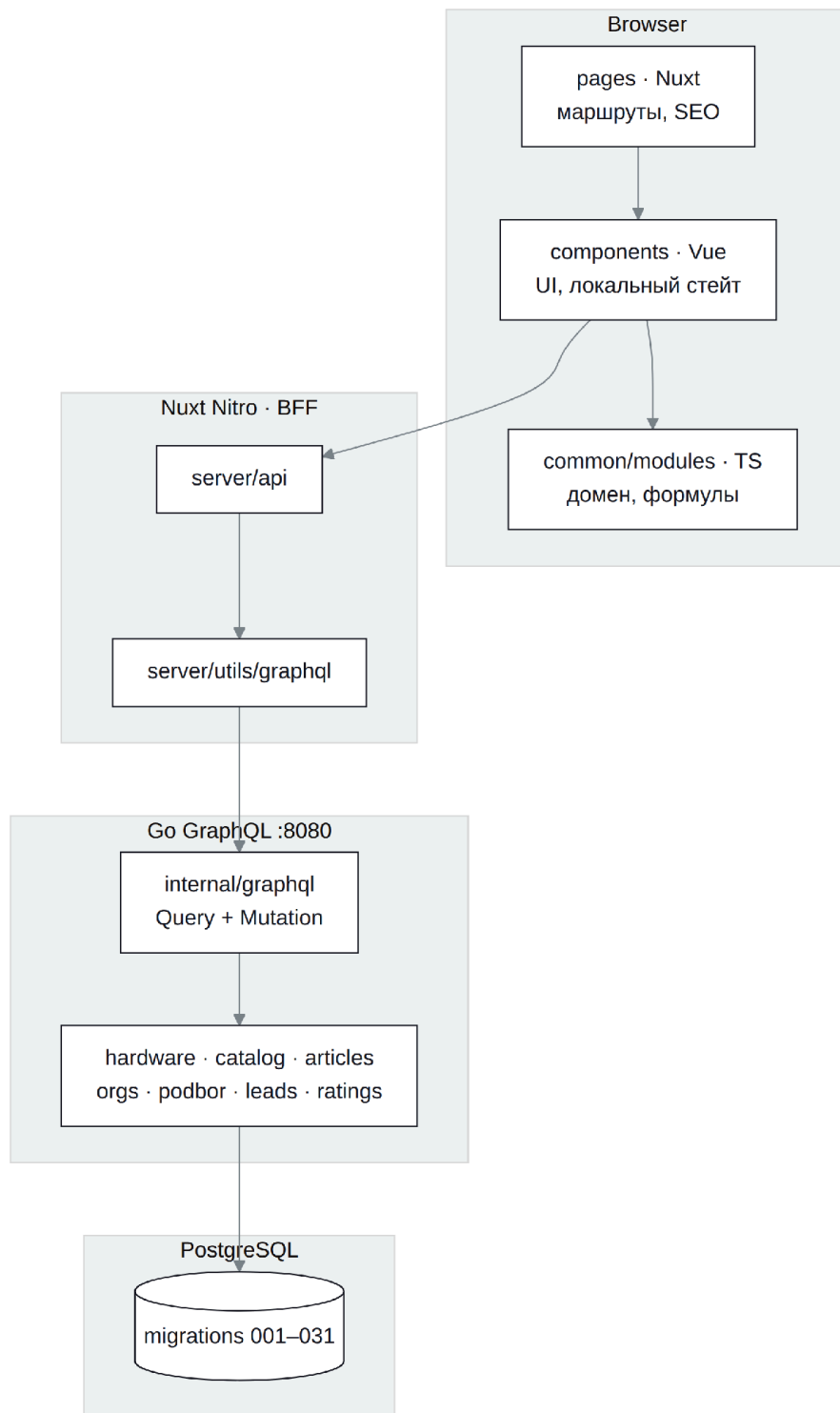
Отправка заявки	12
08 BPMN - бизнес-процесс калькулятора	12
Дорожки (lanes)	13
Основной процесс — по шагам	14
Data Objects	14
Исключения и альтернативные потоки	14
09 Формулы расчёта	15

01 Общая картина системы

layered architecture • component overview

Ключевая архитектурная особенность: **вся математика калькулятора живёт на клиенте**. Бэкенд отдаёт только каталог железа, монет и рыночные дефолты - сам расчёт доходности не делает сетевых запросов. Это делает калькулятор мгновенным: прибыль пересчитывается на каждое изменение поля без обращения к серверу.

СЛОЙ	СТЕК	ОТВЕТСТВЕННОСТЬ
Frontend	Nuxt 3, Vue 3, TS, Quasar, Tailwind, SCSS	UI, локальный стейт форм, клиентский расчёт прибыли
BFF	Nitro server/api/*	HTTP для браузера, Zod-валидация, прокси GraphQL, SMTP / Telegram
Backend	Go + graphql-go	Каталоги, статьи, рейтинги, карточка организации, мутация заявок
DB	PostgreSQL 16 (Docker)	Источник правды для каталога и seed калькулятора



Component diagram — путь запроса через слои

02 Как устроен фронтенд

layers · shell · domains · component diagram

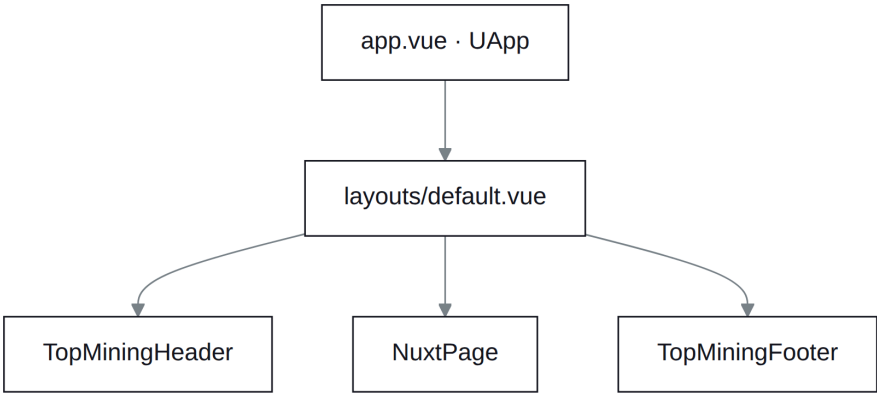
Фронтенд разделён по слоям с жёстким правилом: бизнес-логика не смешивается с разметкой. Разметка живёт в `.vue`, а формулы и типы в чистом TS без Vue.

СЛОЙ	ОТВЕТСТВЕННОСТЬ
pages/	Маршрут, SEO, useFetch, тонкая композиция секций
components/	UI и локальный стейт: открыт дропдаун, активная вкладка
common/modules/	Доменная логика без Vue: типы, формулы, тексты, Zod-схемы
composables/	Auto-import хуки: useSubmitLead, useTopMiningLocale
server/api/	BFF к Go и внешним сервисам

Правило слоёв: не класть бизнес-формулы в `.vue` и не класть разметку в `common/modules/`. Компонент только оркестрирует — берёт данные из модуля и отдаёт их в UI.

Оболочка страницы

Все страницы завернуты в единый каркас: `app.vue` → `layout` → хедер / контент / футер.



Page shell — `app.vue` → `layout` → `header` / `page` / `footer`

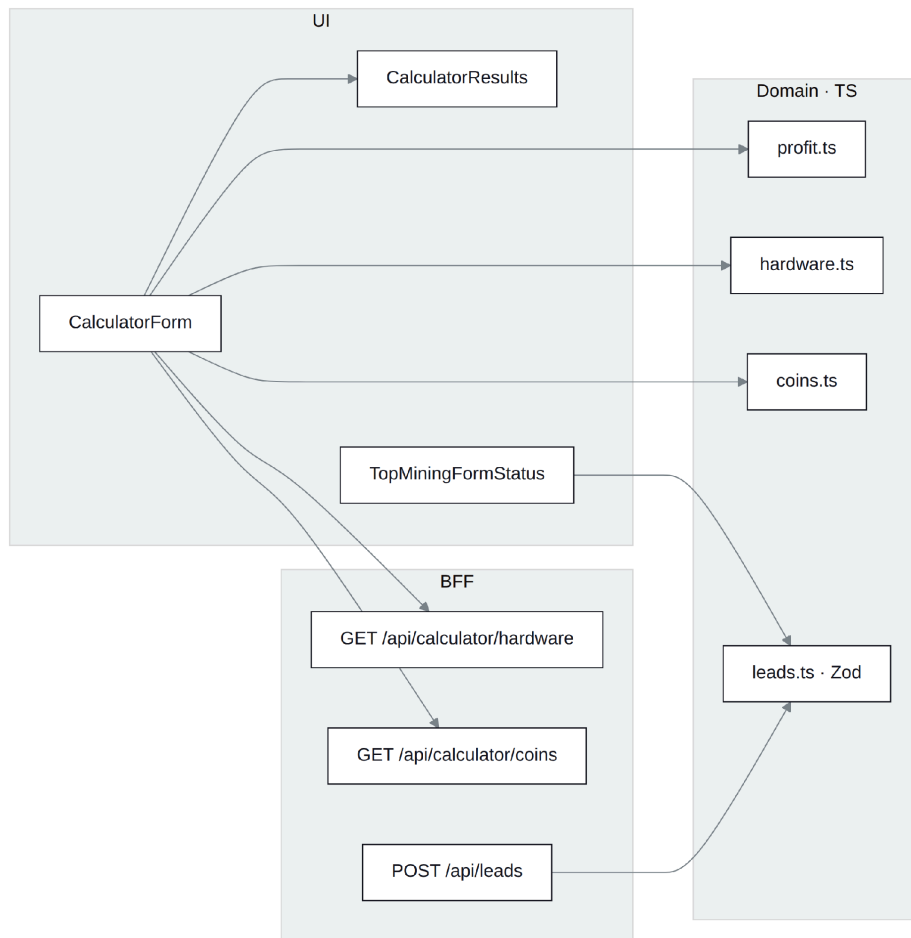
Домены UI

ДОМЕН	ПАПКА КОМПОНЕНТОВ	МОДУЛЬ ЛОГИКИ
Калькулятор	components/calculator/	modules/top-mining/calculator/
Каталог	components/catalog/	modules/catalog/
Статьи	components/articles/	modules/articles/
Рейтинги	components/rating/	modules/ratings/
Подбор / отель	components/podbor/	modules/top-mining/podbor/
Заявки / статус	формы + TopMiningFormStatus	layout/leads.ts, subscribe.ts

ДОМЕН	ПАПКА КОМПОНЕНТОВ	МОДУЛЬ ЛОГИКИ
Хедер / локаль	components/top-mining/header/	layout/locale.ts

Компонентная диаграмма — калькулятор

Форма собирает данные из доменных модулей, вызывает клиентский расчёт и рендерит результаты. Каталог железа и монет подтягивается из BFF при загрузке.



UI ↔ Domain ↔ BFF — связи компонентов калькулятора

03 Как устроен бэкенд

Go · graphql-go · package layout

Бэкенд на Go — это единый GraphQL-сервер на :8080 плюс отдельная утилита миграций. Каждый бизнес-домен вынесен в свой пакет внутри internal/, а схема GraphQL связывает их в одном месте.

структура репозитория

```

backend/
  cmd/server/          # HTTP GraphQL :8080
  cmd/migrate/         # применение SQL
  internal/
    graphql/schema.go  # Query + Mutation
    hardware/          # каталог калькулятора
    catalog/           # категории организаций
    articles/
    organizations/
    ratings/
    podbor/            # placement / sale offers
    leads/             # INSERT заявок
    migrations/        # 001 ... 031

```

ПАКЕТ	ОТВЕТСТВЕННОСТЬ
hardware	Ядро калькулятора: бренды, модели ASIC/GPU/CPU, монеты, алгоритмы, рыночные данные
catalog	Категории и организации каталога
articles	Лента, статья по slug, инкремент просмотров
organizations	Карточка организации и отзывы
ratings	Рейтинги и рейтинги для главной
podbor	Оферты размещения и продажи (майнинг-отель)
leads	INSERT заявки в таблицу leads

04 GraphQL API + UML

schema.go · queries.ts · class diagram

Контракт между фронтом и Go описан в GraphQL. Источник – schema.go, документы операций – server/graphql/queries.ts.

Queries

ДОМЕН	ОПЕРАЦИЯ	НАЗНАЧЕНИЕ
Catalog	catalog	Категории / организации каталога
Calculator	calculatorHardware	Бренды и модели ASIC / GPU / CPU
Calculator	calculatorCoins	Монеты, алгоритмы GPU, defaultUsdtRub
Podbor	podborPlacementOffers	Карточки размещения
Podbor	podborSaleOffers	Карточки продажи
Articles	articlesFeed(topic)	Лента статей
Articles	article(slug)	Статья
Orgs	organization(slug)	Карточка организации

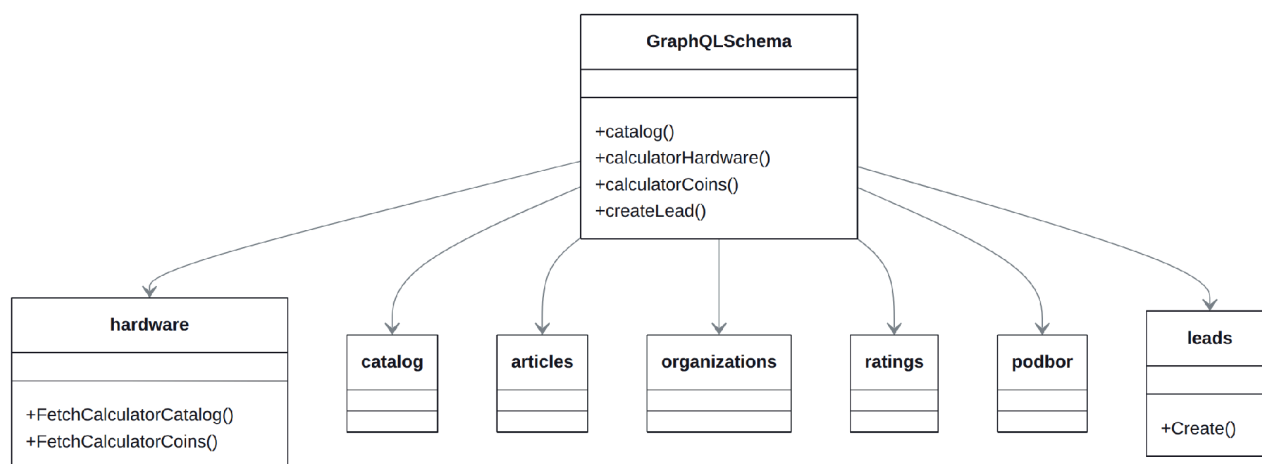
ДОМЕН	ОПЕРАЦИЯ	НАЗНАЧЕНИЕ
Orgs	organizationReviews(slug, sort)	Отзывы
Ratings	ratings, ratingsHome	Рейтинги

Mutations

ДОМЕН	ОПЕРАЦИЯ
Articles	incrementArticleView(slug)
Orgs	createOrganizationReview(...)
Leads	createLead(source, name, contact, message, payload, pagePath)

UML — схема ↔ пакеты Go

Класс-диаграмма показывает, что GraphQL-схема – это фасад: она делегирует каждый резолвер в соответствующий пакет.



Class diagram — GraphQLSchema делегирует в доменные пакеты

05 Nuxt BFF · роуты server/api

HTTP-фасад для браузера

BFF – тонкая HTTP-прослойка. Она превращает REST-запросы браузера в GraphQL к Go, валидирует заявки и подписки через **Zod** и дёргает внешние сервисы (SMTP, Telegram).

ГРУППА	МЕТОД	ПУТЬ
Calculator	GET	/api/calculator/hardware
Calculator	GET	/api/calculator/coins

ГРУППА	МЕТОД	ПУТЬ
Catalog	GET	/api/catalog, /manufacturers, /organizations/[slug]
Catalog	GET/POST	/api/catalog/organizations/[slug]/reviews
Articles	GET	/api/articles, /api/articles/[slug]
Articles	POST	/api/articles/[slug]/view
Ratings	GET	/api/ratings, /api/ratings/home
Podbor	GET	/api/podbor/placement, /api/podbor/sale
Crypto	GET	/api/crypto
Leads	POST	/api/leads
Subscribe	POST	/api/subscribe

Zod-контракты

СХЕМА	ФАЙЛ	ПОЛЯ
leadSubmitSchema	layout/leads.ts	source*, contact*, name, message, fields, website, pagePath
subscribeSubmitSchema	layout/subscribe.ts	email*, source?, website

Обе схемы проверяются дважды – safeParse на клиенте (мгновенная валидация формы) и повторно в BFF (защита от прямых запросов). Поле website – honeypot против ботов.

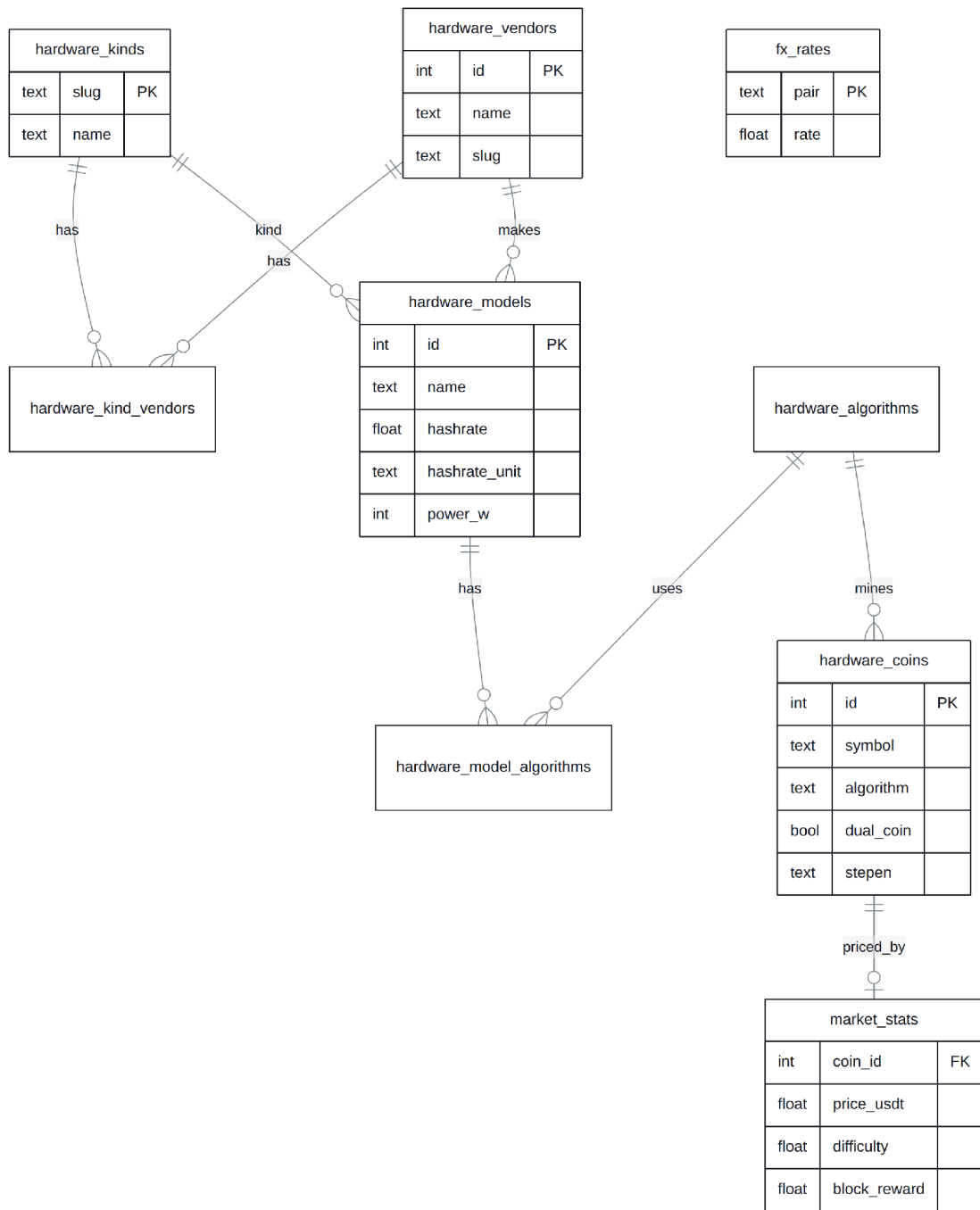
06 Схемы данных · ER

источник правды – migrations/

Единственный источник правды по данным – SQL в backend/migrations/. PlantUML в README упоминается, но актуальны именно миграции.

Ядро калькулятора — hardware

Модель железа связывает виды (hardware_kinds), производителей и модели. Модели умеют работать по разным алгоритмам, алгоритмы майнят монеты, а монеты котируются рыночной статистикой. Курсы валют лежат отдельно в fx_rates.



ER — ядро hardware · миграция 017, сид 028

Ключевые миграции: 017 hardware_domain.sql, сид 028_calculator_catalog_seed.sql (плюс calculator_settings).

Заявки — leads

Плоская таблица. Произвольные данные формы кладутся в payload (jsonb), поэтому одна таблица обслуживает заявки со всех форм сайта.

leads		
bigint	id	PK
text	source	
text	name	
text	contact	
text	message	
jsonb	payload	
text	page_path	
timestampz	created_at	

ER — leads · миграция 031

Прочие домены

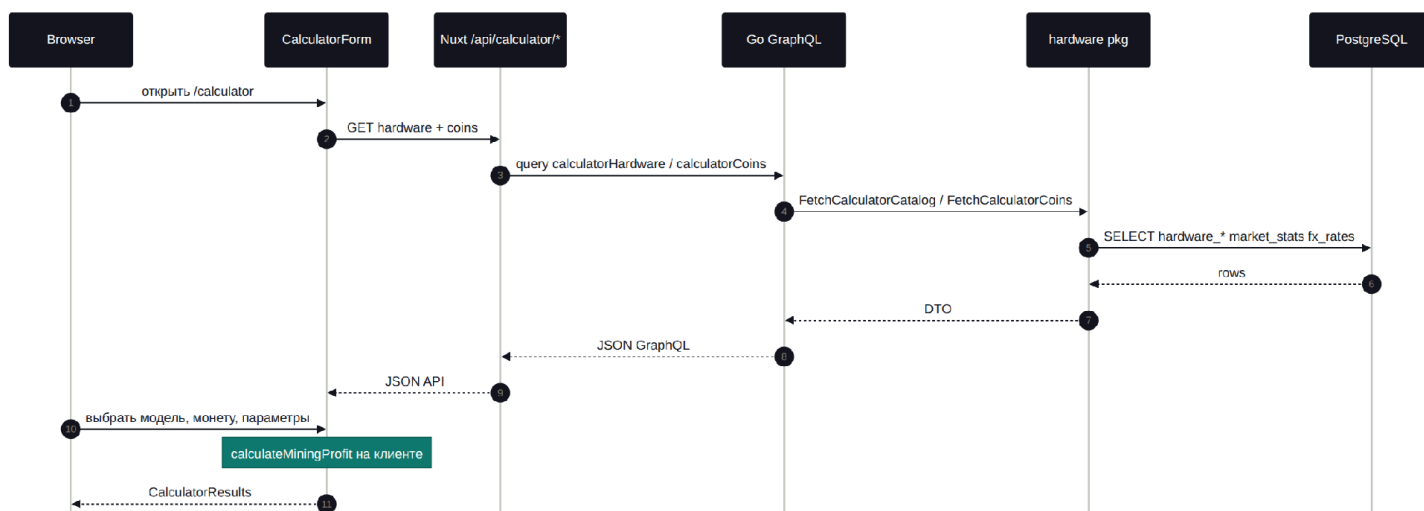
ДОМЕН	МИГРАЦИИ (ОРИЕНТИРЫ)	СУЩНОСТИ
Организации / каталог	003, 010, 025	org, категории, профили
Статьи	002, 018, 027	articles, blocks, views
Рейтинги / отзывы	005, 006, 019	ratings, reviews
Podbor offers	029, 030	placement / sale cards

07 Sequence-диаграммы

каталог калькулятора · отправка заявки

Две диаграммы во времени показывают, как проходят главные сценарии. В первом расчёт прибыли не уходит на сервер вовсе — он замыкается на клиенте.

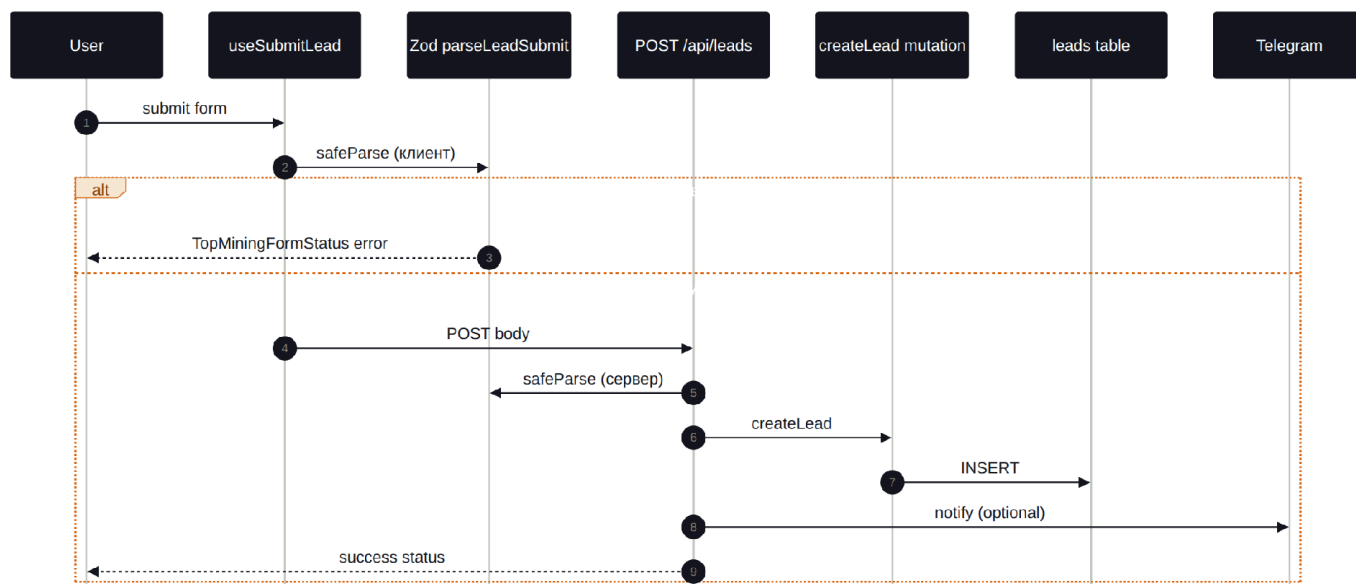
Загрузка каталога калькулятора



Sequence — загрузка hardware и coins, затем расчёт на клиенте

Отправка заявки

Форма валидируется дважды и, при успехе, пишет строку в leads и опционально шлёт уведомление в Telegram.



Sequence — lead submit с двойной Zod-валидацией

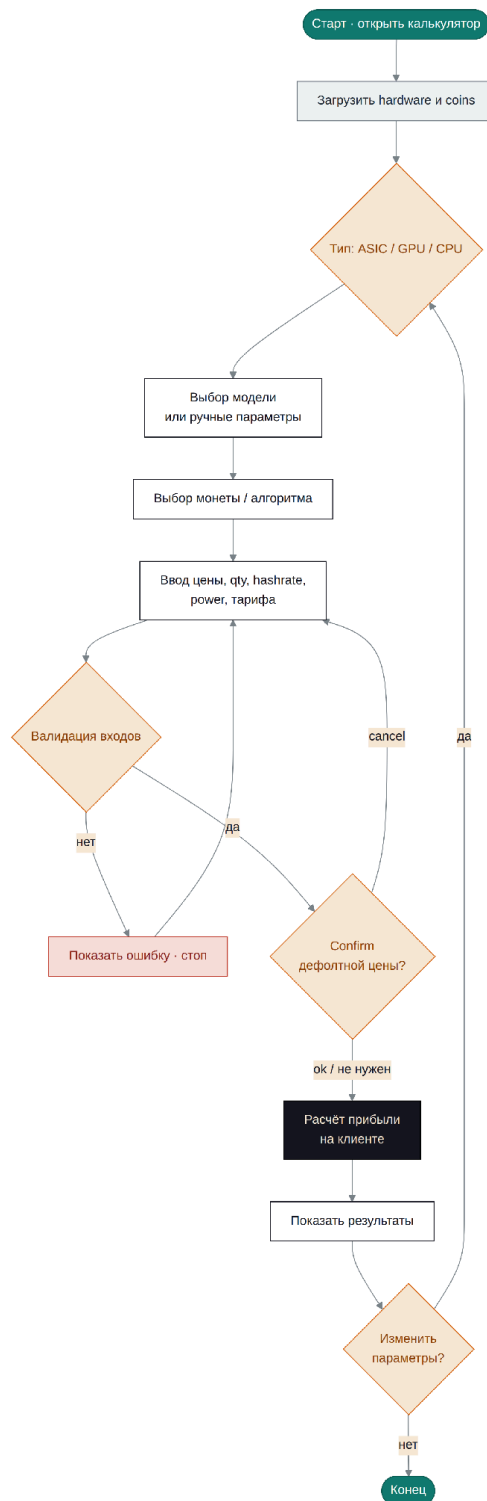
08 BPMN · бизнес-процесс калькулятора

lanes · tasks · gateways · data objects

Модель процесса, удобная для переноса в BPMN 2.0 (Camunda, Bizagi, draw.io). На схеме: прямоугольники – задачи, ромбы – шлюзы (gateways), скруглённые – события старта/конца.

Дорожки (lanes)

Пользователь · UI Калькулятор · Клиентская математика · BFF Nuxt · Backend Go · PostgreSQL.



BPMN процесса калькулятора — happy path и циклы

Основной процесс — по шагам

#	ШАГ (BPMN TASK)	ЧТО ПРОИСХОДИТ
1	Load catalogs	Service task: UI → BFF → Go → DB. Загрузка hardware и coins; пока грузится — скелетоны
2	Select device kind	User task + gateway <i>asic/gpu/cpu</i> . Побочно: сброс модели, для ASIC дефолт BTC
3	Select / unlock equipment	ASIC — модель или ручной ввод; GPU/CPU — модель обязательна. Подставляются <i>hashrate</i> , <i>unit</i> , <i>powerW</i>
4	Select coin / algorithm	ASIC — монета; GPU/CPU — алгоритм, затем отфильтрованные монеты. Dual-coin (LTC+DOG) — два курса
5	Fill economics	Цена, валюта, qty, <i>hashrate</i> , мощность, тариф. Опционально: <i>uptime</i> , <i>pool fee</i> , курсы, <i>reward</i> , <i>difficulty</i>
6	Validate inputs	Business-rule gateway: есть ноги расчёта? <i>hashrate/power/price</i> > 0? Нет — стоп
7	Default price modal	Если цена дефолтная — спросить подтверждение. Cancel возвращает к форме
8	Calculate profit	Script task на клиенте (подпроцесс из формул раздела 09)
9	Show results	Вкладки монета / USDT / ₽: доход, размещение/мес, чистая прибыль, окупаемость
10	End / loop	Пользователь меняет параметры → возврат к шагам 2–5

Data Objects

ОБЪЕКТ	СОДЕРЖИМОЕ
HardwareCatalog	<i>brands</i> → <i>models</i> (<i>hashrate</i> , <i>power</i> , <i>unit</i> , <i>algorithm</i>)
CoinsCatalog	<i>coins</i> , <i>algorithms</i> , <i>defaultUsdtRub</i> , <i>dual</i> -флаги
CalculatorFormState	все поля формы
ProfitLegs	1–2 ноги: <i>reward</i> , <i>difficulty</i> , <i>rate</i> , <i>stepen</i>
ProfitResult	<i>coins*</i> , <i>income*</i> , <i>placingMonth</i> , <i>cleanProfit</i> , <i>paybackMonths</i>

Исключения и альтернативные потоки

СОБЫТИЕ	ПОВЕДЕНИЕ
API hardware/coins недоступен	UI-скелетоны и пустые списки; расчёт без модели ограничен
Dual-coin без курсов	Валидация не пускает в расчёт
Чистая прибыль ≤ 0	<i>paybackMonths</i> = null — «Не окупается»
Смена вкладки ASIC ↔ GPU	Сброс зависимого стейта (прерывание сценария)

09 Формулы расчёта

script task · profit.ts

«script task» из BPMN – вся математика, которая выполняется на клиенте. Полная спецификация: docs/calculator.md.

```
# нормализация и коэффициенты
HR_ths      = hashrate × unitFactor
poolFactor  = (100 - poolFee%) / 100
uptime      = uptimePercent / 100

# добыча монет за период (на одну ногу)
coins_period = (blockReward × HR_ths × 1e12 × 86400 × poolFactor × days × uptime × qty)
              / (difficulty × stepen)

# доход, размещение, прибыль, окупаемость
income_usdt  = Σ (coins_leg × coinUsdtRate_leg)
placing_month_rub = (powerW / 1000) × 732 × uptime × electricityRub × qty
clean_month  = income_month - placing_month
payback_months = deviceCost / clean_month      # если clean_month > 0, иначе null
```

ВЕЛИЧИНА	ПОЯСНЕНИЕ
stepen	Множитель сложности сети: 2v32, 2v13 или 1 — разбирается перед делением на difficulty
732	Среднее число часов в месяце (365 × 24 ÷ 12) для помесечной стоимости электричества
Dual	Для LTC+DOG складываются две ноги: income = leg ₁ + leg ₂ по своим курсам